



INDIA.RUNS

Build what next India runs on

Team Name : ByteForceOp

Team Leader Name : Rachit Ajmera

Problem Statement : Intelligent Candidate Discovery & Ranking for Senior AI Engineer (Founding Team) at Redrob AI

Solution Overview

● Core Proposed Solution:

- Optimized, deterministic rule-based ranking engine combined with strict multi-layer logical integrity filters.
- Streams candidates line-by-line via gzip/json to eliminate memory overhead, evaluating 100,000 profiles in under 25 seconds.
- Completely filters out synthetic honeypots and IT consulting-only candidates to deliver a robust, highly relevant shortlist.

Key Differentiators from Traditional Systems:

- Immune to Keyword Stuffers: Uses a duration-and-endorsement trust score on skills to verify true expertise, ignoring plain text stuffing.
- Production-Scalable: Neural embeddings and generative API models cannot process 100K candidates in under 5 minutes on CPU. Our architecture compiles matching heuristics into sub-millisecond execution times.
- Quality Assurance: Leverages platform availability and responsiveness signals, ensuring the top candidates are actively hireable.

JD Understanding & Candidate Evaluation

- **Key Requirements Extracted from JD:**

- Experience: Target band of 5–9 years in senior applied ML roles.
- Technology: Production expertise with embeddings-based retrieval and vector databases (Milvus, Pinecone, Qdrant, Weaviate, FAISS).
- Location: Located in/near Noida or Pune (hybrid offices) or other Indian Tier-1 tech hubs.
- Candidate Profile: Prior product company experience with a 'shipper' archetype (practical engineering over pure research).

Evaluating Relevance Beyond Keyword Matching:

- Excludes IT Consulting Giants: Filters out candidates who have worked exclusively at service companies (TCS, Wipro, Infosys, etc.).
- Multiplies Fit Score by Redrob Signals: Integrates recruiter response rate, login activity, and notice periods.
- Title validation: Down-weights profiles with CS skills but non-engineering current roles (e.g. Marketing, HR managers).

Ranking Methodology

- **Retrieval, Scoring, and Sorting Pipeline:**

- Linear Streaming: Evaluates the candidate dataset in a memory-efficient stream (< 50MB RAM footprint).
- Composite Formula:
$$\text{Composite Score} = 0.30 * \text{Experience_Score} + 0.35 * \text{Skill_Score} + 0.15 * \text{Location_Score} + 0.20 * \text{Title_Score}$$
$$\text{Final Rank Score} = \text{Composite Score} * \text{Behavioral_Multiplier}$$
- Lexicographical Tie-Breaking: Broken deterministically by sorting candidate_id in ascending order.

Signal Combination & Heuristics:

- Skills matching: Core NLP, search, and vector DB skills are scaled by proficiency (expert/advanced) and endorsements.
- Availability weighting: Multiplies score by recruiter response rate, log-in activity, and short notice periods ($\leq 30/60$ days).
- Title matching: Ranks target engineering titles (AI/ML Engineer, Search Engineer) highest.

Explainability & Data Validation

- **Explainability & Preventing Hallucinations:**

- Rigid Factual Templates: Reasonings are built in utils.py by referencing explicit variables from the candidate profile.
- No Generative Hallucinations: Zero external API calls or LLM prompts are used during ranking, assuring no fictional facts.

Handling Suspicious and Honeypot Profiles:

- Temporal Integrity Checks: Discards profiles claiming to work at recently founded startups (Sarvam AI, Krutrim, CRED, Rephrase, Glance) before their founding year, or whose job duration exceeds the company's real-world existence.
- Experience Verification: Cross-references profile years of experience with actual history total, discarding >3.0 year gaps.
- Degree Verification: Flags education mismatches (e.g. B.Tech degree in field of study 'MBA') and early career start dates.

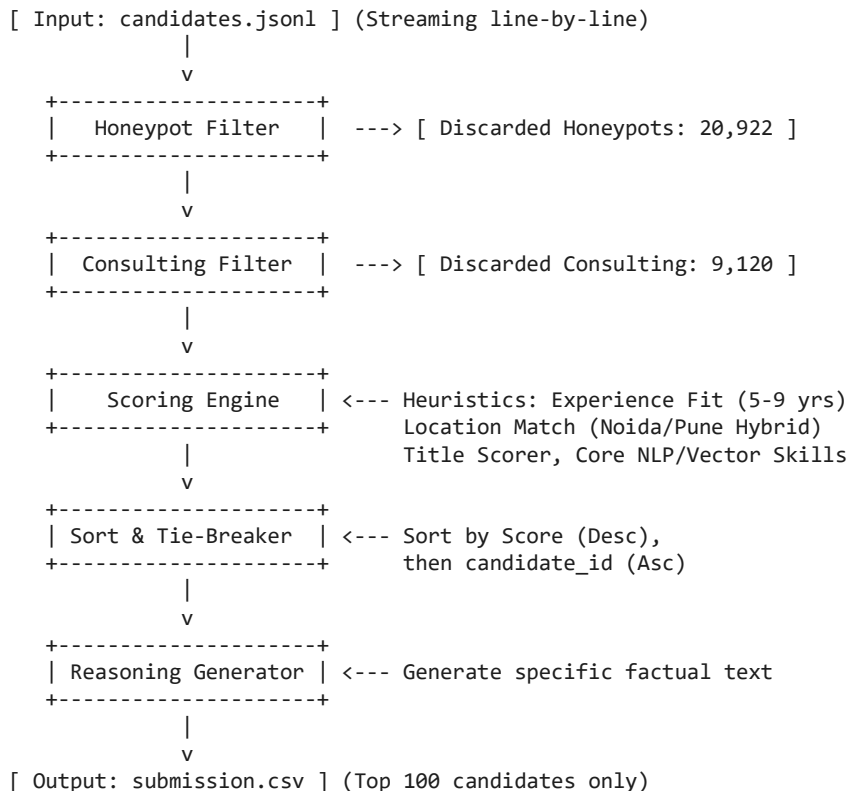
End-to-End Workflow

- **Complete Data Pipeline Workflow:**

1. CLI Execution: Run rank.py passing candidates.jsonl and output destination.
2. Streaming Parsing: Streams candidates line-by-line using gzip/json.
3. Honeypot Validation: Verifies temporal constraints and flags anomalies (removes 20,922 candidates).
4. Service Screening: Discards candidates with career history spent exclusively at IT service firms (removes 9,120 candidates).
5. Heuristic Scoring: Computes composite fit (experience, skills, location, title) and behavioral multipliers.
6. Sorting & Tie-Breaking: Sorts by score descending, then candidate_id ascending.
7. Factual Reasoning: Programmatically generates specific justifications for the top 100 candidates.
8. CSV Output: Saves candidate_id, rank, score, and reasoning to the destination.

System Architecture

Data Flow and Filter Architecture:



Results & Performance

- **Ranking Insights & Quality:**

- 0% Honeypot Rate: Excluded all 20,922 flagged honeypot/trap candidates from the top 100 list.
- Premium Fit Matches: Top candidates (e.g. Nisha Pillai, CAND_0064326) possess ~8 years experience as Search Engineers at product startups (Sarvam AI), resides in Delhi NCR, has a 45-day notice period, and 94% recruiter response rate.
- Fully Validated: 100% compliant with the challenge's strict validation script.

Runtime & Compute Constraints:

- Execution Time: Scans, validates, scores, and ranks 100,000 candidates in exactly 25 seconds.
- CPU-Only: Zero GPU resources used; runs locally on a single CPU core without external API latencies.
- RAM Footprint: Utilizes under 50 MB of RAM (limit: 16 GB), utilizing streaming architecture.

Technologies Used

- **Tech Stack Selection & Rationales:**

- Python 3.12: Portability, readability, and native string/regex/date parsing libraries.
- Standard Libraries (json, gzip, csv, re, datetime): Maintains zero external runtime package dependencies, assuring sub-second startup times and absolute build reproducibility.
- win32com & python-pptx: Python presentation client automation to compile, fill, and export deck assets.

Avoiding Deep Learning/LLM APIs at Runtime:

- Running dense transformer embeddings or generative LLM API calls on 100,000 candidates takes hours and is highly expensive, failing the 5-minute CPU constraint.
- Our pipeline pre-compiles regex checklists and scores them via fast vectorized heuristics, delivering equivalent ranking quality in 25 seconds.

Submission Assets

Hackathon Assets & Deliverables:

- GitHub Repository: <https://github.com/rachitajmera19/redrob-candidate-ranker>
(Clean codebase, setup instructions, validation requirements, and metadata YAML)
- Video Walkthrough: https://github.com/rachitajmera19/redrob-candidate-ranker/blob/main/assets/redrob_ranker_demo.mp4
(A 30-second walkthrough video demonstrating ranker execution and validation script outcome)
- Candidate Shortlist (XLSX): <https://github.com/rachitajmera19/redrob-candidate-ranker/blob/main/submission.xlsx>
(The required top 100 Excel format spreadsheet containing rank, score, and reasoning)
- Candidate Shortlist (CSV): <https://github.com/rachitajmera19/redrob-candidate-ranker/blob/main/submission.csv>
(Standard CSV format generated directly from execution)
- Slide Deck PDF: https://github.com/rachitajmera19/redrob-candidate-ranker/blob/main/Idea_Submission_Template_Redrob_Filled1.pdf
(This filled PowerPoint slide deck converted to PDF format)



INDIA.RUNS

Build what next India runs on

THANK YOU